

2026 年度高 2 情報

Visual Studio 入門

2026/09/26

部品の正体：「オブジェクト」

ここまで画面に配置してきた TextBox や Button は, 全てプログラムから操作可能な「**オブジェクト (モノ)**」である.

オブジェクトを扱う上で, 以下の**2つのキーワード**を押さえておくこと.

1. プロパティ (属性・設定)

その部品が保持するデータ. 色, 大きさ, 表示される文字など.

2. メソッド (動作・命令)

その部品に「やらせる」動作. リセットする, 計算する, 動くなど.

プロパティとメソッドの記述方法

プロパティ（設定）の変更には =（代入）を用いる.

```
// の文字（）に「こんにちは」を代入するLabel1Text  
label1.Text = "こんにちは";  
  
// を押せる状態（）にするButton2Enabled  
button2.Enabled = true;
```

メソッド（動作・命令）の実行には（）をつける.

```
// に「動け」と命令するTimer1  
timer1.Start();
```

※ .（ドット）は「～の」という意味で捉えると解釈しやすい.

アプリ3：10秒連打ゲーム

時間を制御する部品を用いて, 新たなアプリを作成する.

【新しい部品】

- **Timer (タイマー)**: 一定時間ごとに処理を自動実行する不可視の部品. (画面下部の領域に配置される)

【画面の準備】

- Timer1: プロパティの Interval を **1000** (1 秒) に設定する.
- Label1: 残り時間を表示 (プロパティで「10」と入力しておく).
- Label2: スコアを表示 (プロパティで「0」と入力しておく).
- Button1: プロパティの Text を「スタート」にし, 押しやすいように枠を大きく広げておく.

連打ゲームのコード (ボタン)

1つのボタンを「スタート」と「連打」で使い分けるため, if 文で処理を分岐させる.

```
// { } の外に変数を用意 状態を保持するためのデータ()
int time = 10;
int score = 0;
bool isPlaying = false; // ゲーム中かどうかを判定するフラグ
private void button1_Click(object sender, EventArgs e){
    if (isPlaying == false) {
        isPlaying = true;
        button1.Text = "連打! "; // ボタンの文字を書き換える
        time = 10;
        score = 0;
        label1.Text = time.ToString();
        label2.Text = score.ToString();
        timer1.Start(); // タイマー開始
    } else {
        score = score + 1;
        label2.Text = score.ToString();
    }
}
```

連打ゲームのコード（タイマー）

画面下の **Timer1** をダブルクリックし, 1 秒ごとに実行される処理を記述する.

```
// Timer1 秒ごとに呼び出される(1)
private void timer1_Tick(object sender, EventArgs e)
{
    time = time - 1; // 残り時間を減らす
    label1.Text = time.ToString();

    // 秒になったら停止させる0
    if (time == 0)
    {
        timer1.Stop(); // タイマー停止
        isPlaying = false; // ゲーム状態を元に戻す
        button1.Text = "スタート"; // ボタンの文字を元に戻す
        MessageBox.Show("終了！ スコアは " + score + " 回です.");
    }
}
```

アプリ4：ガチャシミュレーター

続いて, データを「リスト」として順次追加する部品を用いる.

【新しい部品】

- **ListBox (リストボックス):** 複数の文字列を一覧表示する枠.

【画面の準備】

- **Button1:** 「1 回引く」 ボタン.
- **Button2:** 「10 連ガチャ」 ボタン.
- **ListBox1:** 結果の履歴を表示する枠 (大きめに配置する).

ガチャのログをリセットするには？

ガチャを引く前に、古い履歴を消す処理

(`listBox1.Items.Clear();`)を追加したい。しかし、これを「1回引く」ボタンの中に書いてしまうと問題が発生する。

10連ガチャのバグ

10連ボタンは「1回引く」を10回繰り返すため、毎回履歴が消去されてしまい、最後の1個しか表示されなくなってしまう。

これを解決するために、「ガチャを1回引いてリストに追加する」という自分だけのメソッド（命令）を新しく作成する。

自分だけの命令 (メソッド) を作る

ボタンをダブルクリックして作られた枠の外側に, 新しい枠
DrawGacha() を自作する.

```
// 自作のメソッド 命令() : ガチャを回引く処理1
Random r = new Random();
private void DrawGacha()
{
    int num = r.Next(100);
    string result = "";

    if (num < 5){
        result = " 【】 SSR 伝説の剣";
    }
    else if (num < 20){
        result = " 【】 SR 鋼の盾";
    }
    else{
        result = " 【】 R 薬草";
    }
    listBox1.Items.Add(result);
}
```

自作メソッドを使ってボタンを書き換える

自作した DrawGacha() を使って, ボタンの処理をスッキリ書き直す.
それぞれのボタンの先頭で Clear() を呼べば, バグは発生しない.

```
// Button1 回引く(1)
private void button1_Click(object sender, EventArgs e)
{
    listBox1.Items.Clear(); // まず履歴をリセット
    DrawGacha();           // 自作した命令を呼び出す
    listBox1.SelectedIndex = listBox1.Items.Count - 1;
}

// Button2 連ガチャ(10)
private void button2_Click(object sender, EventArgs e)
{
    listBox1.Items.Clear(); // まず履歴をリセット
    for(int i = 0; i < 10; i++)
    {
        DrawGacha();       // 自作した命令を回呼び出す10
    }
    listBox1.SelectedIndex = listBox1.Items.Count - 1;
}
```

発展課題：郵便番号検索アプリ

インターネット上のデータ（API）を利用し、郵便番号から住所を自動検索するアプリを作成する。

API (Application Programming Interface)

プログラム同士が連携するための窓口。今回は「zipcloud」という無料APIを利用し、郵便番号のデータを受け取る。

【画面の準備】

- TextBox1: 郵便番号を入力する枠 (例: 1000001).
- Button1: 「住所検索」ボタン.
- Label1: 結果（住所）を表示する場所.

通信のための準備 (おまじない)

インターネット通信やデータの翻訳 (JSON 解析) を行うため, プログラムの一番上に以下の2行を追加する.

```
using System;  
using System.Windows.Forms;  
// ↓以下の行を追加する2  
using System.Net.Http;  
using System.Text.Json;
```

また, 通信待ちの間に「画面がフリーズする」のを防ぐため, ボタンのイベントに **async** というキーワードを追記する.

```
private async void button1_Click(object sender, EventArgs e)  
{  
    // ここに処理を書く  
}
```

郵便番号検索のコード

```
private async void button1_Click(object sender, EventArgs e)
{
    // 入力された郵便番号を末尾に連結する
    string zip = textBox1.Text;
    string url = "https://zipcloud.ibsnet.co.jp/api/search?zipcode=" + zip;

    // 2. インターネットからデータを取得 非同期処理
    HttpClient client = new HttpClient();
    string jsonStr = await client.GetStringAsync(url);

    // 3. 取得したデータから住所を取り出すJSON
    using (JsonDocument doc = JsonDocument.Parse(jsonStr))
    {
        var results = doc.RootElement.GetProperty("results");
        if (results.ValueKind != JsonValueKind.Null) // データが存在した場合
        {
            var item = results[0];
            string pref = item.GetProperty("address1").GetString(); // 都道府県
            string city = item.GetProperty("address2").GetString(); // 市区町村
            string town = item.GetProperty("address3").GetString(); // 町域

            label1.Text = pref + city + town;
        }
        else
        {
            label1.Text = "見つかりませんでした.";
        }
    }
}
```

エラー解決：必要な部品の追加

コードに赤い波線が出た場合、「その機能がまだ準備されていない」というエラーである。今回は `System.Text.Json` や `System.Memory` などの部品を追加する。

拡張機能の導入：NuGet (ヌゲット)

Visual Studio には、足りない機能を無料でダウンロードして追加できる「アプリストア」のような機能（NuGet パッケージ）が備わっている。

実行前に

以下の手順で, 通信や JSON 解析に必要な部品をプロジェクトに追加する.

- ① 上部のメニューから「プロジェクト」→「NuGet パッケージの管理」を選択する.
- ② 左上の「参照」タブをクリックする.
- ③ 検索窓に `System.Text.Json` と入力する.
- ④ 一番上の `System.Text.Json` を選択し, 右側の「インストール」を押す.
- ⑤ 確認画面 (変更のプレビューやライセンスへの同意) が出たら, 「OK」や「同意する」をクリックする.

※これをインストールすると, 関連して必要な `System.Memory` も同様に追加